SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	N.Adi Narayana	**Reg. No.:**	192525346
Date:		**Duration:**	60 minutes

Code of Conduct

I N.Adi Narayana_(192525346)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____Adi___

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or

	justification aligned to system requirements.			justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, unclear, or technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

1. System Objectives

Develop an intelligent, web-based support platform that lets customers submit support tickets in free text and automatically classifies each ticket into a meaningful category using Natural Language Processing (NLP) and Machine Learning (ML). The platform predicts ticket priority, routes every ticket to the appropriate support team, stores all tickets and prediction results, and gives administrators a dashboard to monitor tickets, categories, and classification performance.

The specific objectives of the system are:

- Reduce the manual effort and delay involved in reading and sorting incoming tickets.
- Classify tickets into categories such as Technical Issue, Billing, Account Access, and General Inquiry.
- Identify ticket priority so that urgent issues are handled first.
- Route each ticket to the correct support team automatically.
- Keep a reliable, searchable record of tickets, predicted categories, and classification results.
- Measure and monitor model quality using accuracy, precision, recall, F1-score, and a confusion matrix.
- Allow the classification model to be updated and improved over time without redesigning the system.

2. Functional Requirements

The functional requirements below describe what the system must do. Each requirement is given a priority (Must / Should / Could) so that features can be prioritized during development.

ID	Requirement	Description	Priority
FR1	User registration and login	Customers, support agents, and administrators register and log in with role-based access.	Must
FR2	Ticket submission	Customers submit a ticket (subject, description, optional attachment) through the web interface.	Must
FR3	Text preprocessing	Ticket text is cleaned using tokenization, stop-word removal, and text normalization (lower-casing, punctuation and noise removal, stemming/lemmatization).	Must
FR4	Feature extraction	Preprocessed text is converted into numerical features using TF-IDF.	Must
ID	**Requirement**	**Description**	**Priority**
FR5	Ticket classification	The ML model predicts the category: Technical Issue, Billing, Account Access, or General Inquiry, with a confidence score.	Must
FR6	Priority identification	The system assigns a priority (High, Medium, Low) using urgency keywords, predicted category, and model confidence.	Must
FR7	Ticket routing	Each ticket is assigned to the appropriate support team queue based on category and priority.	Must

FR8	Data storage	Tickets, predicted categories, priorities, routing details, and classification results are stored in the database.	Must
FR9	Ticket tracking	Customers can view the status of their own tickets; agents can update status and add resolution notes.	Should
FR10	Manual category correction	Agents can correct a wrong prediction; corrections are saved as labelled data for retraining.	Should
FR11	Admin dashboard	Administrators view tickets, category distribution, priority levels, team workload, and classification performance.	Must
FR12	Model evaluation	The system computes accuracy, precision, recall, F1-score, and a confusion matrix on test and feedback data.	Must
FR13	Model update	An authorized administrator can retrain the model on new labelled tickets and deploy a new model version.	Should
FR14	Logging and monitoring	Predictions, errors, and user actions are logged for auditing and monitoring.	Should
FR15	Notifications	Customers and agents receive email/SMS notifications for ticket creation, assignment, and status changes.	Could

3. Non-Functional Requirements

Requirement	Description	Influence on Architecture
Performance	Ticket submission and classification should feel immediate to the customer; dashboard pages should load quickly.	Lightweight classical ML model kept in memory; asynchronous processing; indexed queries.
Scalability	Support a growing number of customers and ticket volume.	Stateless API layer and independently scalable classification service.
Availability	The ticket submission service should remain usable with minimal downtime.	Redundant instances, health checks, failover, fallback to manual triage.
Reliability	No ticket may be lost, even if classification fails temporarily.	Tickets are stored before processing; retry mechanism and a pendingclassification state.
Security	Protect customer data; only authorized users may access tickets and dashboards.	Authentication, role-based access control, HTTPS/TLS,

		input validation, data masking in logs.
Maintainability	Easy to modify preprocessing, features, models, and rules.	Modular components with clear interfaces; versioned models; automated tests.
Requirement	**Description**	**Influence on Architecture**
Usability	Simple ticket form for customers and clear charts for administrators.	Separate customer and admin interfaces in the presentation layer.
Interoperability	Connect with email/SMS gateways and external helpdesk or CRM tools.	REST APIs and a dedicated integration boundary.
Data Integrity and Backup	Tickets and results must be stored accurately and be recoverable.	Relational database with constraints, replication, and scheduled backups.

4. Quality Attributes

Scalability: Support an increasing number of customers, tickets, and support teams, and the addition of new ticket categories.

Performance: Fast text preprocessing, feature extraction, and prediction so that classification adds little delay to ticket submission.

Reliability: Consistent and correct storage of tickets and predictions; tickets are never lost when a component fails.

Availability: Continuous (24×7) access for submitting tickets and for support teams to receive routed work, with fault tolerance.

Maintainability: Independent modules (preprocessing, features, classifier, routing) that can be updated or replaced without affecting the rest of the system.

Security: Authentication, authorization, encrypted communication, input validation, and audit logs to protect customer information.

5. Selected Software Architecture

The proposed solution adopts a Hybrid Architecture that combines a Layered Architecture with Modular Services. The layered structure separates the presentation layer, the application/API layer, the ticketprocessing (intelligence) layer, and the database layer. Within the processing layer, the NLP Preprocessing, Feature Extraction, ML Classification, Priority and Routing, and Model Evaluation and Monitoring modules are built as loosely coupled services with well-defined interfaces. The ML classification module can be deployed as a separate service, so it can be scaled and updated independently of the web application. This architecture is simpler than full microservices but still provides flexibility, scalability, and easy maintenance.

6. Architecture Overview

- **Presentation Layer:** Customer web interface (ticket submission and tracking) and Admin web interface/dashboard (tickets, categories, performance metrics) for customers, support agents, and administrators.

- **API / Application Layer:** API Gateway for authentication, request validation, and routing, together with the Ticket Management Module that creates, updates, assigns, and tracks tickets.
- **Ticket Processing (Intelligence) Layer:** NLP Preprocessing Module, Feature Extraction Module (TF-IDF), Machine Learning Classification Module, and Priority and Routing Module.
- **Model Evaluation and Monitoring Module:** Computes accuracy, precision, recall, F1-score, and confusion matrix, tracks logs, and supports model updates.
- **Database Layer:** Relational database for tickets, categories, priorities, routing, and users; results and audit log storage; and a model artifact store for trained models and TF-IDF vocabularies.
- **External Systems:** Email/SMS gateway for notifications, optional helpdesk/CRM integration, and an identity/SSO provider.

7. Justification for Selecting the Architecture

- The ticket classification task is naturally a pipeline (preprocess, extract features, classify, prioritize, route). A layered design with modular pipeline stages maps directly onto this flow.
- Separating the ML classification module allows it to be scaled and retrained independently of the customer-facing web application.
- Each stage (for example, replacing TF-IDF with word embeddings or Naive Bayes with SVM) can be changed without affecting the other layers.
- Failures are isolated: if the classifier is unavailable, tickets are still saved and queued for manual triage.
- Centralized authentication and validation in the API layer improve security.
- It is realistic to build and operate for a student or small-team project, avoiding the overhead of a full microservices system.
- It supports future expansion such as deep-learning models, multilingual tickets, chatbots, and CRM integration.

The table below compares candidate architectural styles for this problem.

Architectural Style	Assessment for this Project	Suitability
Layered Architecture	Clear separation of UI, logic, processing, and data; easy to understand and test.	Very good (core style)
MVC	Good for the web interface but does not describe the NLP/ML processing pipeline.	Partial (used inside the web layer)
Client–Server	Basic fit, but gives no structure to the internal processing components.	Partial
Full Microservices	Maximum independence but high operational complexity for a system of this size.	Over-engineered
Event-Driven	Useful for asynchronous processing, but complex for the current scope.	Optional future use

Serverless	Cost-effective for bursts, but model loading delays and vendor dependence are drawbacks.	Limited
Hybrid: Layered + Modular Services (selected)	Layered structure for clarity plus independently deployable ML/processing modules.	Best fit

8. Advantages

Feature	Benefit
Scalability	The API layer and the classification service scale independently as ticket volume grows.
Performance	A lightweight TF-IDF plus linear model gives fast prediction; asynchronous processing keeps submission responsive.
Reliability	Tickets are stored first; failure of the classifier does not lose tickets.
Maintainability	Modular design makes it easy to update preprocessing rules, features, or models.
Feature	**Benefit**
Integration	REST APIs allow easy connection to email/SMS gateways, helpdesk/CRM tools, and identity providers.
Security	Central authentication, role-based access control, and encrypted communication protect data.
Transparency	Stored predictions, confidence scores, and metrics make the system easy to audit and monitor.
Future Readiness	Supports advanced NLP models, retraining with feedback, and cloud deployment.

9. Software Architecture Diagram

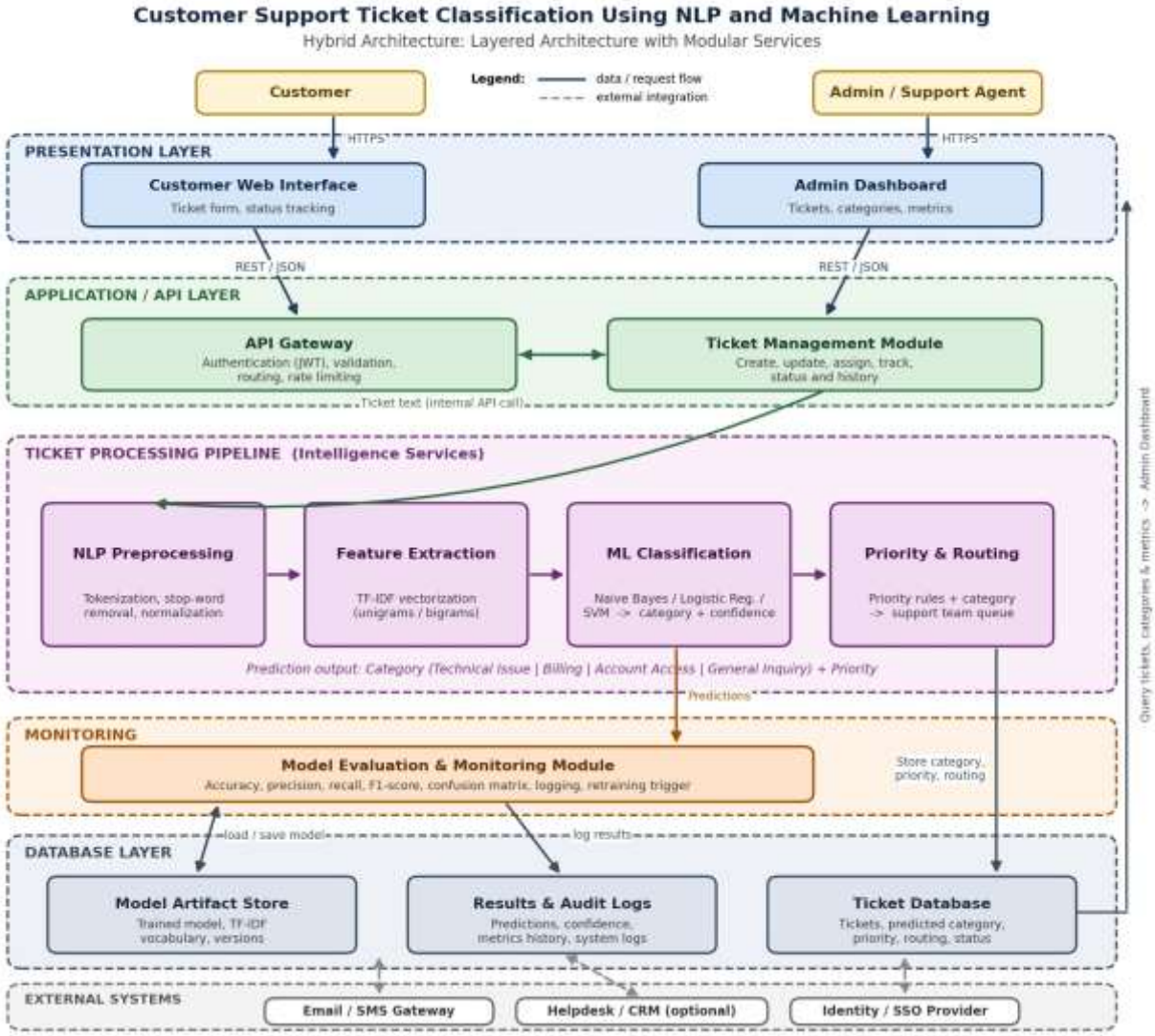


Figure 1: Software architecture of the Customer Support Ticket Classification system

10. Diagram Description

The architecture follows a Hybrid Layered + Modular Services approach. Customers and administrators/support agents access the system through the web interfaces in the Presentation Layer over HTTPS. Requests are sent as REST/JSON calls to the API / Application Layer, where the API Gateway authenticates and validates them and the Ticket Management Module creates and manages tickets. The ticket text is then passed to the Ticket Processing Pipeline: NLP Preprocessing, Feature Extraction (TFIDF), ML Classification, and Priority and Routing. The predicted category, priority, and routing decision are stored in the Database Layer. The Model Evaluation and Monitoring Module receives predictions and feedback, computes the performance metrics, and uses the model artifact store and the logs. The Admin Dashboard retrieves tickets, categories, and metrics from the database. External systems such as the email/SMS gateway, helpdesk/CRM, and identity provider are connected through the application layer.

Notation used in the diagram:

- Dashed rounded rectangles represent architectural layers; solid rounded boxes inside them represent components or modules.

- Solid arrows show the direction of request and data flow; double-headed arrows show two-way communication.
- Dashed arrows show integration with external systems.
- Labels on arrows (HTTPS, REST/JSON, internal API call) show the communication mechanism.

11. Major Components

- Presentation Layer (Customer Web Interface and Admin Dashboard)
- API Gateway
- Ticket Management Module
- NLP Preprocessing Module
- Feature Extraction Module (TF-IDF)
- Machine Learning Classification Module
- Priority and Routing Module
- Model Evaluation and Monitoring Module
- Database Layer (Ticket Database, Results and Audit Logs, Model Artifact Store)
- External Systems (Email/SMS Gateway, Helpdesk/CRM, Identity/SSO Provider)

12. Architectural Components and Responsibilities

Presentation Layer: Provides the Customer Web Interface for registering, logging in, submitting tickets, and tracking ticket status, and the Admin Web Interface/Dashboard for viewing tickets, category and priority distribution, team workload, and model performance. It contains no business logic and communicates only with the API layer.

API Gateway / Application Layer: Acts as the single entry point. It authenticates users (token-based), applies role-based access control, validates and sanitizes input, rate-limits requests, and routes each request to the correct module.

Ticket Management Module: Creates tickets, assigns unique IDs, stores the initial record before processing, manages ticket status (New, Classified, Assigned, In Progress, Resolved), accepts agent corrections, and returns ticket data to the interfaces.

NLP Preprocessing Module: Cleans the raw ticket text. It performs tokenization, stop-word removal, and text normalization (lower-casing, removal of punctuation, URLs, numbers and special characters, and stemming or lemmatization) so that the text is consistent for feature extraction. The same preprocessing steps are used for training and prediction.

Feature Extraction Module: Converts the preprocessed text into numerical vectors using TF-IDF (optionally with unigrams and bigrams). The TF-IDF vocabulary learned during training is saved with the model so that new tickets are transformed in the same way.

Machine Learning Classification Module: Loads the trained model and predicts the ticket category (Technical Issue, Billing, Account Access, General Inquiry) with a confidence score. Candidate algorithms are Multinomial Naive Bayes, Logistic Regression, and Linear Support Vector Machine; the final algorithm is selected by comparing their validation metrics on the labelled dataset. Predictions with low confidence are flagged for manual review.

Algorithm	Strengths	Considerations

Naive Bayes	Very fast, simple, works well as a baseline on text.	Assumes feature independence; may be less accurate on overlapping categories.
Logistic Regression	Good accuracy on TF-IDF features; gives probability scores useful for confidence thresholds.	Needs regularization tuning.
Support Vector Machine (Linear)	Strong performance on high-dimensional sparse text data.	Does not give probabilities directly (calibration needed).

Priority and Routing Module: Determines ticket priority (High, Medium, Low) using urgency keywords (for example “urgent”, “cannot login”, “charged twice”, “system down”), the predicted category, model confidence, and optionally the customer type. It then routes the ticket to the appropriate team queue, for example Technical Support, Billing Team, Account/Security Team, or General Customer Service. Routing rules are kept in configuration so they can be changed without modifying the code.

Model Evaluation and Monitoring Module: Evaluates the classifier using a held-out test set and agentcorrected tickets. It computes the metrics shown in the table below and monitors logs and performance trends. It also supports model updates by triggering retraining when enough new labelled tickets are available or when performance drops. No performance figures are claimed in this report; the values will be produced when the model is trained and tested on the actual labelled dataset.

Metric	Meaning	Use in this Project
Accuracy	Fraction of tickets classified correctly.	Overall correctness of the classifier.
Precision	Of the tickets predicted as a category, how many were correct.	Avoids sending wrong tickets to a team.
Recall	Of the tickets that truly belong to a category, how many were found.	Ensures important tickets are not missed.
F1-score	Harmonic mean of precision and recall (per class and macro/weighted).	Balanced measure, useful when categories are imbalanced.
Confusion Matrix	Table of actual versus predicted categories.	Shows which categories are confused (for example Billing and Account Access).

The proposed evaluation method is to split the labelled data into training, validation, and test sets (stratified by category), use cross-validation for model selection, and report per-category and macro-averaged metrics along with the confusion matrix.

Database Layer: Stores users and roles, tickets, predicted categories, priorities, routing and status (Ticket Database); predictions with confidence scores, evaluation metrics history, and system/audit logs (Results and Audit Logs); and trained model files, TF-IDF vocabularies, and version information (Model Artifact Store). A relational database provides integrity constraints, indexing, and reporting queries.

External Systems: An email/SMS gateway sends notifications; an optional helpdesk/CRM integrates with existing support tools; an identity/SSO provider supports secure login. They are accessed through REST APIs over HTTPS.

13. Component Interaction

1. The customer opens the web interface, logs in, and submits a ticket (subject and description).
2. The API Gateway authenticates the request, validates and sanitizes the input, and forwards it to the Ticket Management Module.
3. The Ticket Management Module saves the new ticket in the Ticket Database with status "New" and returns a ticket ID to the customer.
4. The ticket text is sent to the NLP Preprocessing Module, which tokenizes the text, removes stop words, and normalizes it.
5. The cleaned tokens are passed to the Feature Extraction Module, which creates the TF-IDF feature vector using the saved vocabulary.
6. The ML Classification Module loads the current model from the Model Artifact Store and predicts the category and confidence score.
7. The Priority and Routing Module combines the category, confidence, and urgency keywords to assign a priority and selects the support team queue.
8. The predicted category, priority, routing, and confidence are stored in the Ticket Database; the prediction details are stored in the Results and Audit Logs.
9. The Ticket Management Module updates the ticket status to "Assigned", and the Notification integration sends confirmation and assignment alerts through the email/SMS gateway.
10. Support agents view their queue, work on the ticket, and may correct the category; the correction is stored as labelled feedback.
11. The Model Evaluation and Monitoring Module periodically reads predictions and feedback, computes accuracy, precision, recall, F1-score, and the confusion matrix, and stores the results.
12. The Admin Dashboard retrieves tickets, category distribution, team workload, and performance metrics from the database through the API layer.

The communication mechanisms between components are summarized below.

Interaction	Mechanism	Data Exchanged
User → Web Interface → API Gateway	HTTPS, REST/JSON	Login credentials, ticket form data, dashboard requests.
API Gateway ↔ Ticket Management	Internal function/API call	Validated ticket data, ticket status, query results.
Ticket Management → NLP → Features → Classifier	Internal service calls (synchronous or via a task queue)	Ticket text, cleaned tokens, TF-IDF vectors, predicted category and confidence.
Classifier → Priority and Routing	Internal call	Category, confidence, urgency indicators; returns priority and team.

Modules → Database Layer	SQL / database driver	Tickets, predictions, routing, logs, model versions.
Classifier ↔ Model Artifact Store	File/object read and write	Trained model and TF-IDF vocabulary.
Application Layer ↔ External Systems	REST APIs over HTTPS	Notifications, identity tokens, optional helpdesk/CRM records.
Database → Admin Dashboard	REST API queries	Ticket lists, category counts, evaluation metrics.

14. Overall System Workflow

Initially, the customer logs in and submits a support ticket through the web interface. The system stores the ticket and sends its text through the NLP Preprocessing Module, where it is tokenized, cleaned of stop words, and normalized. The Feature Extraction Module converts the text into a TF-IDF vector, and the trained Machine Learning model predicts the ticket category, such as Technical Issue, Billing, Account Access, or General Inquiry, along with a confidence score. The Priority and Routing Module then assigns a priority and routes the ticket to the appropriate support team. Tickets with low confidence are marked for manual triage so that wrong automatic routing is avoided.

The prediction, priority, and routing details are saved in the database and the customer and team are notified. Support agents resolve the ticket and can correct the category if the prediction was wrong. These corrections, together with the logged predictions, are used by the Model Evaluation and Monitoring Module to calculate accuracy, precision, recall, F1-score, and the confusion matrix, which are displayed on the Admin Dashboard. When performance drops or enough new labelled tickets are collected, the model is retrained offline using the same preprocessing and TF-IDF steps, evaluated, and deployed as a new version through the Model Artifact Store.

The offline model lifecycle can be summarized as follows:

Stage	Activity
1. Data collection	Collect historical and corrected tickets with category labels.
2. Preprocessing	Tokenization, stop-word removal, normalization.
3. Feature extraction	Fit the TF-IDF vectorizer on the training data.
4. Training	Train Naive Bayes, Logistic Regression, and/or SVM candidates.
5. Evaluation	Compute accuracy, precision, recall, F1-score, and the confusion matrix on validation and test data.
6. Deployment	Store the selected model and vocabulary as a new version in the Model Artifact Store.
7. Monitoring	Track live predictions, agent corrections, and logs; trigger retraining when needed.

15. Quality Attribute Analysis

The proposed Hybrid Layered Architecture with Modular Services is evaluated against the key quality attributes required for a Customer Support Ticket Classification system. These attributes ensure that the system remains efficient, secure, reliable, and ready for future growth.

1. Scalability

The API layer is stateless, so more application instances can be added behind a load balancer as the number of users grows. The ML Classification Module is a separate service and can be scaled independently when ticket volume rises. A task queue can absorb bursts of tickets, and read replicas or cached aggregates can support dashboard queries. New categories or support teams can be added by retraining the model and updating routing configuration.

2. Security

The API Gateway enforces authentication and role-based authorization so that customers see only their own tickets and only administrators can view dashboards or retrain models. All communication uses HTTPS/TLS. Passwords are stored as salted hashes, input is validated and sanitized to prevent injection attacks, and database access uses parameterized queries. Sensitive customer details are masked in logs, and audit logs record important actions.

3. Performance

TF-IDF with a linear classifier is computationally lightweight, so prediction is fast. The trained model and vocabulary are loaded into memory once and reused for each request. Asynchronous processing keeps ticket submission responsive, and database indexes on ticket ID, status, category, and date speed up dashboard queries. Frequently requested dashboard summaries can be cached.

4. Reliability

The ticket is saved before any processing starts, so it is never lost if a later stage fails. If classification fails, the ticket stays in a “Pending Classification” state and is retried or sent to manual triage. Automated database backups, replication, health checks, and retry mechanisms support dependable operation.

5. Maintainability

Each module (preprocessing, feature extraction, classification, routing, evaluation) has a clear interface and a single responsibility, so it can be changed and tested independently. Models and vocabularies are versioned, routing rules and priority keywords are stored in configuration, and automated unit and integration tests, logging, and documentation reduce maintenance effort.

6. Availability

Multiple application and classification instances, load balancing, database replication, and failover mechanisms support continuous operation. Continuous monitoring and health checks detect failures early. Because of failure isolation, the ticket submission and tracking functions remain available even when the classifier or the dashboard is temporarily unavailable.

Summary of architectural tactics:

Quality Attribute	Key Architectural Tactics
Scalability	Stateless API, independent classification service, task queue, load balancing, read replicas.
Security	API gateway authentication, RBAC, HTTPS/TLS, input validation, salted password hashing, audit logs, log masking.

Performance	Lightweight TF-IDF + linear model, in-memory model, asynchronous processing, indexing, caching.
Reliability	Store-before-process, retries, pending state, manual triage fallback, backups.
Maintainability	Modular components, versioned models, configurable rules, automated tests, CI/CD.
Availability	Redundant instances, failover, replication, health monitoring, failure isolation.

16. Justify Architectural Decisions

1. Rationale Behind the Architectural Decisions

The proposed system adopts a Hybrid Architecture that combines Layered Architecture with Modular Services to meet the requirements of a Customer Support Ticket Classification system. The layered structure separates the presentation, application/API, ticket-processing, and database layers, which makes the application easy to understand, test, and maintain. The ticket-processing pipeline is split into NLP Preprocessing, Feature Extraction, ML Classification, and Priority and Routing modules, because each has a different reason to change: preprocessing rules, feature techniques, algorithms, and business routing rules evolve independently. Classical ML (TF-IDF with Naive Bayes, Logistic Regression, or SVM) is chosen because it is fast, requires modest data and hardware, and is easy to explain, which suits a first version of the system.

2. Trade-offs Considered

Decision	Advantage	Trade-off
Hybrid Layered + Modular Services	Clear structure with independent scaling of the ML module.	More design effort than a single-block application.
Modular services instead of full microservices	Lower operational complexity and cost.	Less independence than fully separate microservices.
TF-IDF with classical ML instead of deep learning	Fast, low-cost, explainable, works with smaller datasets.	May capture less context and meaning than transformer models.
Separate ML classification service	Model can be updated and scaled without redeploying the web app.	Extra network call and service coordination.
Rule-based priority identification	Transparent, quick to configure and change.	Rules need maintenance and may miss unusual phrasing.
API Gateway	Centralized security and routing.	Additional network hop and a potential single point of failure unless made redundant.
Store ticket before classification	No ticket is lost if processing fails.	Tickets may briefly exist without a category.
Asynchronous classification	Responsive submission under heavy load.	Category appears slightly after submission.

Relational database	Integrity, reporting queries, and mature tooling.	Less flexible for unstructured data than a document store.
Human feedback and retraining	Model improves over time using real corrections.	Needs agent effort and careful validation before deployment.

3. Support for Future Enhancements

- Replacing TF-IDF and linear models with word embeddings or transformer models (for example BERT) by changing only the feature extraction and classification modules.
- Support for multilingual tickets with language detection and language-specific preprocessing.
- Sentiment and urgency detection to improve priority identification.
- Automatic suggested replies, FAQ matching, and chatbot integration for common issues.
- Additional ticket channels such as email, chat, and social media ingested through the API layer.
- Active learning, drift detection, and automated retraining pipelines for continuous improvement. ● Explainable predictions, for example showing the top terms that influenced a category.
- Containerization (Docker/Kubernetes) and cloud deployment for automated scaling and orchestration.

4. System Integration

The proposed architecture integrates with external systems through REST APIs and secure HTTPS communication. These include email/SMS gateways for notifications, identity/SSO providers for authentication, helpdesk or CRM platforms (for example for creating or syncing tickets), and cloud services for hosting and storage. Because the ML classification service exposes a clear API, it can also be reused by other applications, such as a chatbot or a mobile app, enabling efficient data exchange and future expansion.

5. Long-Term Maintainability

The modular architecture allows each module to be developed, tested, deployed, and upgraded independently. Versioned models and vocabularies make it possible to roll back a poor model release. Continuous Integration and Continuous Deployment (CI/CD), automated testing, logging, monitoring, and comprehensive documentation improve software quality and reduce maintenance effort. Configurationbased routing rules and priority keywords allow support managers' needs to change without code changes.

6. Overall Justification

The Hybrid Layered Architecture with Modular Services is the most appropriate solution for the Customer Support Ticket Classification system because it satisfies the functional and non-functional requirements while delivering scalability, security, performance, reliability, availability, and maintainability. Its pipelineoriented modular design matches the NLP and ML workflow, keeps the system practical to build and operate, supports future innovations such as advanced language models, and provides a sustainable foundation for long-term software evolution.

Conclusion

The proposed software architecture addresses the challenge of handling growing volumes of customer support tickets by automatically classifying them using NLP and Machine Learning. By combining a layered design with modular services, the system achieves a balance between simplicity, flexibility, and scalability. The architecture covers ticket submission, text

preprocessing, TF-IDF feature extraction, category prediction, priority identification, routing, storage, an admin dashboard, and model evaluation and monitoring, and it supports model updates and future enhancements. It is well suited to present operational needs and future technological advancements, making it a reliable solution for intelligent customer support ticket management.